

# Lean 4 与 mathlib 自学讲义

## 从可执行定义、小型证明到可审计 verifier

RUCmonk2

2026 年 8 月

# 如何使用这本讲义

本讲义面向第一次系统学习 Lean 4、同时希望把它用于 AI4Math、AI4TCS 和随机算法形式化的读者。目标不是背 tactic，而是建立一条稳定链路：把自然语言命题写成精确类型，读懂 proof state，构造能被 kernel 检查的证明项，并诚实记录尚未形式化的语义与性能性质。

建议每个例子都完成四次检查：

1. 在纸上写出对象、假设和结论，不从 tactic 开始；
2. 用 `#check` 确认关键表达式的类型；
3. 在固定 toolchain 的 Lake 项目中编译，而不只依赖编辑器提示；
4. 故意删掉一个假设或改错一个类型，观察 Lean 的错误信息如何变化。

代码示例以 Lean 4 与 mathlib 的常见写法为主。mathlib API 会随版本演进，学习项目必须提交 lean-toolchain 与 Lake 配置；当示例和本机版本不一致时，应先检索当前版本的定义和定理，而不是盲目改到“没有红线”。

# 目录

|                                  |           |
|----------------------------------|-----------|
| <b>如何使用这本讲义</b>                  | <b>1</b>  |
| <b>第一章 环境、Lake 项目与内核检查模型</b>     | <b>5</b>  |
| 1.1 Lean 工具链由什么组成 . . . . .      | 5         |
| 1.2 创建最小 Lake 项目 . . . . .       | 5         |
| 1.3 第一个可检查文件 . . . . .           | 6         |
| 1.4 从源码到 kernel 的路径 . . . . .    | 6         |
| 1.5 错误信息先按层分类 . . . . .          | 7         |
| 1.6 最小可复现环境记录 . . . . .          | 8         |
| <b>第二章 表达式、定义、函数与计算</b>          | <b>9</b>  |
| 2.1 每个表达式都有类型 . . . . .          | 9         |
| 2.2 函数类型与函数定义 . . . . .          | 9         |
| 2.3 显式参数、隐式参数与命名参数 . . . . .     | 10        |
| 2.4 局部定义、命名空间与模块 . . . . .       | 10        |
| 2.5 模式匹配与递归 . . . . .            | 10        |
| 2.6 归约与定义性相等 . . . . .           | 11        |
| 2.7 可执行不等于已证明 . . . . .          | 11        |
| 2.8 定义设计检查表 . . . . .            | 12        |
| <b>第三章 命题即类型与基础证明</b>            | <b>13</b> |
| 3.1 Curry–Howard 对应 . . . . .    | 13        |
| 3.2 直接证明与 tactic proof . . . . . | 13        |
| 3.3 蕴含、全称量词与引入 . . . . .         | 14        |
| 3.4 合取、析取与分类讨论 . . . . .         | 14        |
| 3.5 存在量词必须给出 witness . . . . .   | 14        |
| 3.6 否定、矛盾与底类型 . . . . .          | 14        |
| 3.7 等式、重写与简化 . . . . .           | 15        |

|            |                                   |           |
|------------|-----------------------------------|-----------|
| 3.8        | 常用 tactic 的逻辑角色 . . . . .         | 15        |
| 3.9        | 构造逻辑与经典逻辑 . . . . .               | 16        |
| <b>第四章</b> | <b>归纳类型、递归与归纳证明</b>               | <b>17</b> |
| 4.1        | 归纳类型由构造子生成 . . . . .              | 17        |
| 4.2        | 模式匹配定义函数 . . . . .                | 17        |
| 4.3        | 自然数归纳 . . . . .                   | 18        |
| 4.4        | 列表归纳 . . . . .                    | 18        |
| 4.5        | 树归纳与局部假设 . . . . .                | 18        |
| 4.6        | 先找不变量，再做归纳 . . . . .              | 19        |
| 4.7        | 归纳假设太弱怎么办 . . . . .               | 19        |
| 4.8        | 良基递归与终止证明 . . . . .               | 20        |
| 4.9        | 归纳证明的审计问题 . . . . .               | 20        |
| <b>第五章</b> | <b>依赖类型、结构、子类型与等式</b>             | <b>21</b> |
| 5.1        | 返回类型可以依赖输入 . . . . .              | 21        |
| 5.2        | 无效状态不可表示，还是允许后证明 . . . . .        | 21        |
| 5.3        | structure: 命名字段的乘积类型 . . . . .    | 21        |
| 5.4        | 一个保持不变量的完整小定理 . . . . .           | 22        |
| 5.5        | Subtype: 值与性质打包 . . . . .         | 22        |
| 5.6        | Sigma 类型与存在命题 . . . . .           | 23        |
| 5.7        | 等式与 dependent transport . . . . . | 23        |
| 5.8        | Typeclass: 由类型驱动接口搜索 . . . . .    | 23        |
| 5.9        | 可判定性与计算 . . . . .                 | 24        |
| <b>第六章</b> | <b>mathlib 检索、自动化与证明调试</b>        | <b>25</b> |
| 6.1        | 先读目标，再搜名字 . . . . .               | 25        |
| 6.2        | 目标形状比自然语言关键词更可靠 . . . . .         | 25        |
| 6.3        | 建议型搜索 tactic . . . . .            | 25        |
| 6.4        | 常见领域自动化 . . . . .                 | 26        |
| 6.5        | 控制 simp 的作用域 . . . . .            | 26        |
| 6.6        | 类型不匹配的诊断 . . . . .                | 27        |
| 6.7        | 最小失败例子 . . . . .                  | 27        |
| 6.8        | 不要用过宽 import 掩盖依赖 . . . . .       | 28        |
| <b>第七章</b> | <b>算法不变量、有限随机性与证明分层</b>           | <b>29</b> |

|            |                                       |           |
|------------|---------------------------------------|-----------|
| 7.1        | 先把随机性变成显式输入 . . . . .                 | 29        |
| 7.2        | 状态机、不变量与后置条件 . . . . .                | 29        |
| 7.3        | 有限均匀样本空间 . . . . .                    | 30        |
| 7.4        | indicator 与期望 . . . . .               | 30        |
| 7.5        | 有限 union bound 的计数骨架 . . . . .        | 31        |
| 7.6        | 随机算法保证的六层拆分 . . . . .                 | 31        |
| 7.7        | Count-Min 的分层形式化示例 . . . . .          | 31        |
| 7.8        | 实现与规范的连接 . . . . .                    | 32        |
| 7.9        | 建议的第一个随机算法形式化题 . . . . .              | 32        |
| <b>第八章</b> | <b>Lean 在 AI4Math 与 AI4TCS 中的证明工程</b> | <b>34</b> |
| 8.1        | Lean 在闭环中的位置 . . . . .                | 34        |
| 8.2        | AI4Math 的四类任务 . . . . .               | 34        |
| 8.3        | statement fidelity 是独立任务 . . . . .    | 35        |
| 8.4        | proof search 的状态、动作与反馈 . . . . .      | 35        |
| 8.5        | proof repair 的不变约束 . . . . .          | 35        |
| 8.6        | AI4TCS 中的算法验证 . . . . .               | 36        |
| 8.7        | 证据登记表 . . . . .                       | 36        |
| 8.8        | 数据集与污染问题 . . . . .                    | 36        |
| 8.9        | 失败轨迹也是研究产物 . . . . .                  | 37        |
| 8.10       | 个人可完成的最小闭环 . . . . .                  | 37        |
| <b>第九章</b> | <b>综合习题与参考答案</b>                      | <b>39</b> |
| 9.1        | 习题 . . . . .                          | 39        |
| 9.2        | 参考答案与提示 . . . . .                     | 40        |

# 第一章 环境、Lake 项目与内核检查模型

## 1.1 Lean 工具链由什么组成

第一次接触 Lean 时，容易把 VS Code 中的彩色目标窗口当成 Lean 本身。实际上常用 workflow 至少包含四层：

**elan** 安装并切换 Lean toolchain，作用类似 Rust 的 `rustup`；

**Lean** 解析、展开和检查 Lean 源文件，最终由小型 kernel 验证证明项；

**Lake** 管理项目、依赖和构建任务；

**编辑器扩展** 把当前位置的目标、诊断和补全展示出来，但它不是最终可信边界。

先在终端记录真实版本：

```
elan --version  
lean --version  
lake --version
```

同一个项目应固定 `lean-toolchain`，复制代码时也要带上 `toolchain` 与依赖版本。缺少这些信息时，“在他那里能编译”不能推出“在当前环境能编译”。

## 1.2 创建最小 Lake 项目

在空目录中可以运行：

```
lake init LeanSelfStudy  
lake build
```

不同 Lean 版本生成的模板可能略有差别，但应至少识别以下对象：

| 文件或目录                                                   | 作用                   |
|---------------------------------------------------------|----------------------|
| <code>lean-toolchain</code>                             | 固定 Lean 版本；构建复现的第一入口 |
| <code>lakefile.toml</code> 或 <code>lakefile.lean</code> | 声明包、库、可执行程序 and 依赖   |
| <code>Main.lean</code>                                  | 模板中的可执行入口，若项目需要命令行程序 |
| <code>LeanSelfStudy/</code>                             | 自己的库模块；文件路径通常对应模块名   |
| <code>lake-manifest.json</code>                         | Lake 解析后的依赖锁定信息      |
| <code>.lake/</code>                                     | 本机下载与构建缓存，不应当作学习成果   |

引入 mathlib 时，应按 mathlib 当前文档创建项目或添加依赖，再运行 `lake update` 和 `lake build`。不要手工复制 mathlib 的缓存目录；项目配置和 manifest 才是可审计依赖。

## 1.3 第一个可检查文件

把下面内容放进项目模块中：

```
import Mathlib

#check Nat
#check Nat.add
#check Nat.add_comm

def double (n : Nat) : Nat := n + n

#eval double 6

example (n : Nat) : double n = n + n := by
  rfl
```

`#check` 让 elaborator 告诉我们表达式的类型；`#eval` 对可执行定义求值；`example` 声明一个无需命名保存的定理。最后的 `rfl` 能成功，是因为展开 `double` 后两边定义性相同。

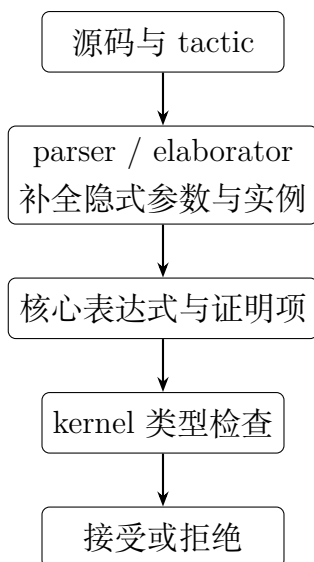
单独检查文件和检查整个项目分别使用：

```
lake env lean LeanSelfStudy/Basic.lean
lake build
```

前者适合缩小错误，后者负责确认模块依赖和整个项目都一致。

## 1.4 从源码到 kernel 的路径

Lean 不直接把你键入的 tactic 字符串交给 kernel。可以用下图理解处理链：



Tactic、搜索器甚至语言模型都可以帮助构造证明项；最终 kernel 只检查这个项是否具有声明的类型。由此得到两个同时成立的判断：

1. 自动化很强不必然扩大可信计算基；只要最终产生普通证明项，kernel 仍会复查；
2. kernel 接受只保证“形式语句被证明”，不保证自然语言题目被正确翻译，也不保证定理有研究意义。

#### 与 AI4Math / AI4TCS 的连接

AI proof agent 可以生成 statement、tactic 或完整 proof term，Lean 负责形式正确性。人类仍需检查 statement fidelity、依赖中是否引入过强公理、问题是否被偷换，以及算法复杂度等未进入命题的性质。

## 1.5 错误信息先按层分类

面对失败时，先问它发生在哪一层：

**解析错误** 括号、缩进、关键字或 Unicode/ASCII 记号不合法；

**名称解析错误** 没有导入模块、命名空间不对或 API 已改名；

**elaboration 错误** 隐式参数、类型类实例或预期类型无法确定；

**证明目标未关闭** tactic 执行过，但仍有 goals；

**构建错误** 单文件似乎可用，但模块名、依赖或项目配置不一致。

#### 容易混淆

重启编辑器、删除全部缓存偶尔能解决环境问题，却不能替代错误分类。若一个 10 行例子仍然失败，应先保存完整错误和 proof state，再最小化代码；否则一次“偶然变绿”不会留下可复用理解。



## 1.6 最小可复现环境记录

一个可以交给别人复查的 Lean 结果至少应包含：

1. Lean 与 mathlib 的固定版本；
2. Lake 配置、manifest 和所有必要源文件；
3. 从干净 checkout 开始的构建命令；
4. 定理是否使用 `axiom`、`sorry` 或非标准代码生成路径；
5. 失败时的最小例子和完整错误，而不是只有截图。

### 本章验收

1. 解释 `elan`、`Lean`、`Lake`、编辑器扩展各自负责什么；
2. 创建一个固定 `toolchain` 的项目，并用终端完成一次 `lake build`；
3. 说明 `tactic`、`elaborator`、`proof term` 与 `kernel` 的关系；
4. 把一次失败归类为解析、名称、`elaboration`、证明或构建错误；
5. 说清 `Lean kernel` 接受一条定理能保证什么、不能保证什么。

## 第二章 表达式、定义、函数与计算

### 2.1 每个表达式都有类型

Lean 的核心判断写成

$$\Gamma \vdash t : T,$$

表示在局部环境  $\Gamma$  中，项  $t$  具有类型  $T$ 。例如自然数 3 的类型是 `Nat`，函数 `Nat.succ` 的类型是 `Nat -> Nat`，而一个等式证明也是某个命题类型的项。

```
#check 3
#check Nat.succ
#check fun n : Nat => n + 1
#check Type
#check Prop
```

类型本身也需要类型。Lean 使用 universe 层级避免“所有类型组成的类型包含自身”带来的悖论。入门时可以先记住：`Nat : Type`，而 `Type : Type 1`；多态定义经常让 universe 参数由 elaborator 自动补全。

### 2.2 函数类型与函数定义

若输入类型为  $A$ 、输出类型为  $B$ ，函数类型写作  $A \rightarrow B$ 。匿名函数和命名定义分别写作：

```
#check fun n : Nat => n * n

def square (n : Nat) : Nat := n * n

def compose (f : B -> C) (g : A -> B) : A -> C :=
  fun x => f (g x)
```

最后一个定义中的 `A B C` 需要预先作为类型变量声明：

```
universe u v w
variable {A : Type u} {B : Type v} {C : Type w}
```

函数默认柯里化。 $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$  解析为  $\text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat})$ ：先接受一个自然数，再返回一个接受自然数的函数。因此多参数应用写作  $f\ x\ y$ ，而不是必须构造元组。

## 2.3 显式参数、隐式参数与命名参数

圆括号参数必须显式提供，花括号参数通常由上下文推断：

```
def identity {A : Type} (x : A) : A := x

#check identity 5
#check identity "lean"
#check @identity
```

`@identity` 要求显示包括隐式参数在内的完整函数。诊断类型推断时，还可以用命名参数指定：`identity (A := Nat) 5`。

### 容易混淆

“参数没有写出来”不等于“参数不存在”。mathlib 定理常含 universe、类型、实例和假设等多层隐式参数。应用失败时先 `#check` 定理，再必要时查看 `@theoremName` 或打开更详细的 pretty printer。

## 2.4 局部定义、命名空间与模块

`let` 创建局部名称；`namespace` 避免全局命名冲突：

```
namespace SelfStudy

def twice (f : Nat -> Nat) (x : Nat) : Nat :=
  let y := f x
  f y

#eval twice (fun n => n + 1) 10

end SelfStudy
```

跨文件使用定义时，需要让文件路径、模块名与 `import` 对应。`open` 只改变名称访问方式，并不会导入尚未加载的模块。

## 2.5 模式匹配与递归

代数数据类型通过构造子产生值，模式匹配负责拆解值：

```
def isZero : Nat -> Bool
| 0 => true
| _ + 1 => false

def sum : List Nat -> Nat
| [] => 0
| x :: xs => x + sum xs

#eval sum [2, 3, 5]
```

`sum` 的递归调用作用于严格更小的尾列表，所以 Lean 的终止检查器可以接受。逻辑一致性要求一般递归不能无条件地产生任意类型的值；若递归不是结构递减，需要提供终止度量与证明。

### 直觉

函数式定义同时承担三种角色：它是可执行程序，是数学对象，也是后续证明展开的重写规则。定义接口若写得清楚，计算和证明会共享同一份结构；定义若混入过多实现细节，后续 `theorem` 也会被这些细节绑定。

## 2.6 归约与定义性相等

Lean 中两类“相等”需要分开：

**定义性相等** 通过展开定义、函数应用归约或模式匹配计算后相同，`kernel` 可直接识别；

**命题相等** 类型为 `Eq a b` 的命题，需要一个证明项。

例如：

```
def addOne (n : Nat) := n + 1

example : addOne 2 = 3 := by
  rfl

example (n : Nat) : n + 0 = n := by
  simp using Nat.add_zero n
```

第一条可通过计算完成；第二条对变量  $n$  并不是简单把两边算成数字，需要自然数加法的定理（该定理本身由归纳证明）。

## 2.7 可执行不等于已证明

`#eval` 能展示某个输入的计算结果，但它没有证明对所有输入成立。例如运行排序函数得到有序列表，只验证了一个样例。一般性正确性需要分别说明：输出有序、输出是输

入的排列，以及函数终止。

### 与 AI4Math / AI4TCS 的连接

AI4TCS 中模型可以生成候选程序, `#eval`、单元测试和随机测试用于快速筛选; `Lean theorem` 用于证明明确写入类型的性质。测试与证明共享定义，却提供不同等级的证据，不能互相冒充。

## 2.8 定义设计检查表

在开始证明前，先检查：

1. 输入、输出和错误情况是否在类型中表达清楚；
2. 使用 `Nat`、`Int`、`Fin n` 还是一般类型是否符合问题；
3. 无效状态能否被构造，若能，后续需要什么不变量；
4. 定义是面向规范还是面向高性能实现，两者是否需要分层；
5. 递归为何终止，模式是否覆盖所有构造子。

### 本章验收

1. 用 `#check` 解释五个表达式的完整类型；
2. 写出一个带隐式类型参数的多态函数，并用 `@` 查看完整参数；
3. 定义列表乘积或最大值的安全版本，说明空列表怎样处理；
4. 区分定义性相等与需要证明的命题相等；
5. 解释为什么 `#eval` 的十个样例不能代替全称定理。

## 第三章 命题即类型与基础证明

### 3.1 Curry–Howard 对应

Lean 中命题位于 `Prop`，证明命题  $P$  就是构造一个类型为  $P$  的项。常见逻辑连接词对应类型构造：

| 逻辑                | Lean 形式                             | 证明数据                               |
|-------------------|-------------------------------------|------------------------------------|
| $P \Rightarrow Q$ | <code>P -&gt; Q</code>              | 把 $P$ 的证明映到 $Q$ 的证明的函数             |
| $P \wedge Q$      | <code>And P Q</code>                | 同时含 $P$ 与 $Q$ 证明的结构                |
| $P \vee Q$        | <code>Or P Q</code>                 | 左分支或右分支，并携带相应证明                    |
| $\forall x, P(x)$ | <code>forall x, P x</code>          | 对每个 $x$ 返回证明的依赖函数                  |
| $\exists x, P(x)$ | <code>Exists fun x =&gt; P x</code> | 一个 witness 和它满足性质的证明               |
| $\neg P$          | <code>Not P</code>                  | 从 $P$ 的证明导出 <code>False</code> 的函数 |

这一观点把“证明步骤”变成精确的构造操作，而不是自然语言中模糊的“显然”。

### 3.2 直接证明与 tactic proof

同一个定理可以直接写 proof term，也可以用 tactic 逐步构造：

```
example (p q : Prop) (hp : p) (hq : q) : And p q :=
  And.intro hp hq

example (p q : Prop) (hp : p) (hq : q) : And p q := by
  constructor
  exact hp
  exact hq
```

第二段执行 `constructor` 后产生两个目标。编辑器展示的 proof state 可以抽象成：

$$\underbrace{p, q : \text{Prop}, hp : p, hq : q}_{\text{local context}} \vdash \underbrace{p \wedge q}_{\text{target}}$$

每个 tactic 都应当被理解为如何改变这个 judgment。

### 3.3 蕴含、全称量词与引入

证明函数类型时，引入输入：

```
example (p q : Prop) : p -> q -> p := by
  intro hp
  intro hq
  exact hp

example : forall n : Nat, n = n := by
  intro n
  rfl
```

`intro` 不是“假设结论成立”，而是按照函数类型构造一个接受证明或对象的函数。

### 3.4 合取、析取与分类讨论

合取的使用是拆字段，析取的使用必须覆盖两个分支：

```
example (p q : Prop) (h : And p q) : And q p := by
  cases h with
  | intro hp hq => exact And.intro hq hp

example (p q : Prop) (h : Or p q) : Or q p := by
  cases h with
  | inl hp => exact Or.inr hp
  | inr hq => exact Or.inl hq
```

`cases` 按 `And.intro` 构造子拆出两个字段。Lean 也支持更紧凑的结构模式，但它不会减少需要提供的的数据。

### 3.5 存在量词必须给出 witness

```
example : Exists fun n : Nat => n + 1 = 4 := by
  refine Exists.intro 3 ?_
  rfl
```

`refine` 提供带“洞”的部分证明，`?_` 变成新目标。存在性证明的核心是 `witness 3`，而不是 `tactic` 的名字。

### 3.6 否定、矛盾与底类型

`False` 没有构造子；一旦得到它，可以推出任意命题：

```
example (p : Prop) (hp : p) (hnp : Not p) : False := by
  exact hnp hp

example (p q : Prop) (h : p -> q) (hnq : Not q) : Not p := by
  intro hp
  exact hnq (h hp)
```

第二条定理的目标 `Not p` 展开就是 `p -> False`，所以先 `intro hp`。

## 3.7 等式、重写与简化

若有  $h : a = b$ ，`rw [h]` 按方向把  $a$  替换为  $b$ ；`rw [<- h]` 反向替换。`simp` 使用标记过的简化规则反复重写：

```
example (a b c : Nat) (h : a = b) : a + c = b + c := by
  rw [h]

example (xs : List Nat) : [] ++ xs = xs := by
  simp
```

`rw` 更适合表达“使用了哪条等式”；`simp` 更适合清理单位元、构造子和已注册规则。遇到失败时，先明确期望哪一处发生哪种替换。

## 3.8 常用 tactic 的逻辑角色

| tactic                   | 主要作用                        |
|--------------------------|-----------------------------|
| <code>intro</code>       | 为函数、蕴含或全称目标引入变量/假设          |
| <code>exact</code>       | 提供类型与当前目标完全匹配的证明项           |
| <code>apply</code>       | 反向使用定理，把当前目标变成它的前提          |
| <code>constructor</code> | 使用目标类型的构造子并产生字段目标           |
| <code>cases</code>       | 按归纳类型或析取的构造子分支讨论            |
| <code>rcases</code>      | 用模式拆解存在、合取或结构               |
| <code>rw</code>          | 按给定等式定向重写                   |
| <code>simp</code>        | 用简化规则集做规范化重写                |
| <code>have</code>        | 声明局部中间结论，控制证明结构             |
| <code>show</code>        | 重新表述当前目标，帮助 elaboration 或阅读 |



## 3.9 构造逻辑与经典逻辑

Lean 的核心逻辑是构造性的。排中律、双重否定消去等经典原理可以显式导入或局部使用，但应让依赖可见：

```
example (p : Prop) : Or p (Not p) := by
  classical
  exact Classical.em p
```

形式化数学中使用经典逻辑通常完全合理；重要的是不要把“可构造 witness”与“经典地证明存在”混为一谈，尤其当后续希望从证明提取算法时。

### 容易混淆

Tactic 成功不等于你已经理解定理。至少应能回答：当前目标是什么、tactic 使用了哪个构造子或定理、产生了哪些子目标、是否引入经典公理，以及删除哪个假设后证明会失败。

### 本章验收

1. 分别用 proof term 和 tactic 证明合取交换；
2. 证明  $(p \rightarrow q) \rightarrow (\text{Not } q \rightarrow \text{Not } p)$ ，逐行解释 proof state；
3. 构造一个自然数 witness，证明某个简单存在命题；
4. 用 `rw` 与 `simp` 各完成一个等式证明，并比较两者；
5. 找出一个需要经典逻辑的命题，明确标出 `classical` 的作用域。

## 第四章 归纳类型、递归与归纳证明

### 4.1 归纳类型由构造子生成

自然数、列表、树和语法树都可以看成“由有限组构造子生成的最小类型”。例如一个二叉树：

```
inductive BinTree (A : Type) where
  | empty : BinTree A
  | node : BinTree A -> A -> BinTree A -> BinTree A
deriving Repr
```

这个声明同时提供：构造值的方法、按构造子分类讨论的原则，以及递归/归纳时可使用的 eliminator。任何 `BinTree A` 都由 `empty` 或 `node` 有限构造而来。

### 4.2 模式匹配定义函数

```
namespace BinTree

def size : BinTree A -> Nat
  | .empty => 0
  | .node left _ right => size left + 1 + size right

def mirror : BinTree A -> BinTree A
  | .empty => .empty
  | .node left x right => .node (mirror right) x (mirror left)

end BinTree
```

每个分支对应一个构造子。Lean 检查模式是否覆盖所有情况，并检查递归调用是否作用于结构上更小的子项。

#### 直觉

归纳类型给出“数据能怎样产生”，递归定义给出“怎样消费数据得到一个值”，归纳证明给出“怎样对所有可能的产生方式证明性质”。三者共享同一个构造子结构。

## 4.3 自然数归纳

要证明  $P(n)$  对所有  $n : \mathbb{N}$  成立，需要证明  $P(0)$ ，以及从  $P(n)$  推出  $P(n+1)$ 。例如：

```
theorem add_zero_self (n : Nat) : n + 0 = n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    simp [Nat.add, ih]
```

这里 `ih` 的类型是前一自然数上的归纳假设。实际项目通常直接使用 `mathlib` 已有的 `Nat.add_zero`；手工证明的目的在于看清归纳原则，而不是重复造库。

## 4.4 列表归纳

列表只有空表与 `cons` 两种构造：

```
theorem append_nil_self (xs : List A) : xs ++ [] = xs := by
  induction xs with
  | nil => rfl
  | cons x xs ih =>
    simp [ih]
```

在 `cons` 分支，归纳假设只覆盖尾列表 `xs`。若结论涉及额外参数，参数是否在归纳前引入，会影响归纳假设的强弱。

## 4.5 树归纳与局部假设

二叉树节点有两个递归子树，所以节点分支获得两个归纳假设：

```
namespace BinTree

theorem size_mirror (t : BinTree A) :
  size (mirror t) = size t := by
  induction t with
  | empty => rfl
  | node left x right ihLeft ihRight =>
    simp [mirror, size, ihLeft, ihRight,
      Nat.add_assoc, Nat.add_comm, Nat.add_left_comm]

end BinTree
```

证明的数学内容是：镜像交换左右子树，但加法可交换，因此节点总数不变。`simp` 负责展开定义并使用归纳假设；交换律和结合律负责规范化加法顺序。

## 4.6 先找不变量，再做归纳

算法证明中的归纳对象不一定是一个数字，也可能是：

- 输入列表或语法树；
- 已处理的流前缀；
- 状态转换的步数；
- 执行关系的推导树；
- 递归调用的结构或终止度量。

假设顺序处理输入  $x_1, \dots, x_n$ ，状态满足

$$\text{Inv}(s_i, x_1, \dots, x_i).$$

典型证明由初始化与保持性组成：

$$\text{Inv}(s_0, []), \quad \text{Inv}(s_i, p_i) \Rightarrow \text{Inv}(\text{step}(s_i, x_{i+1}), p_{i+1}).$$

然后对前缀或步数归纳，得到所有执行阶段的不变量。

## 4.7 归纳假设太弱怎么办

常见症状是：进入归纳分支后，`ih` 只对某个固定参数成立，而目标需要它对变化后的参数成立。处理顺序应当是：

1. 在纸上写出真正需要的加强命题；
2. 决定哪些参数应在归纳前保持一般；
3. 必要时用 `generalizing` 或先 `revert` 参数；
4. 只有命题强度正确后，才继续尝试自动化。

例如证明带累加器的尾递归函数正确时，通常应证明“对任意初始累加器”的更强命题，而不是只证明累加器为零的特例。

### 容易混淆

反复尝试 `simp`、`aesop` 不能修复错误的归纳命题。若归纳假设在数学上不足，搜索更多 `tactic` 只会掩盖真正问题：需要加强 `theorem statement` 或改变归纳对象。

## 4.8 良基递归与终止证明

结构递归之外，欧几里得算法、快速排序或图搜索常按数值度量下降。Lean 允许为递归提供终止依据，例如列表长度、区间大小或一个良基关系。证明需要明确每次调用的度量严格减小。

对学习项目，建议先把函数拆成：

1. 纯粹的一步转换；
2. 显式 fuel 的有限执行器；
3. 关于 fuel 或度量的单调/终止 lemma；
4. 最终的部分正确性与终止性结论。

这比一开始把复杂递归、状态和正确性揉进一个定义更容易调试。

## 4.9 归纳证明的审计问题

阅读任何归纳证明时都问：归纳对象是什么？构造子是否覆盖完全？每个归纳假设对应哪一个递归子对象？命题是否为了归纳而加强？证明只得到部分正确性，还是也包含终止性？

### 与 AI4Math / AI4TCS 的连接

语言模型生成归纳证明时，最常见的失败并不是不会写 tactic，而是选错归纳变量或忘记泛化参数。一个 verifier 应记录失败 proof state 和归纳假设形状，让搜索器能够修改 statement/induction schema，而不只是继续采样 tactic。

### 本章验收

1. 自定义一个带叶子值的树，并写 size 与 map；
2. 对列表证明一个结构归纳定理，指出 base 与 step；
3. 证明树镜像两次得到原树，标出两个归纳假设；
4. 举出一个需要加强归纳命题的累加器例子；
5. 区分结构递归、良基递归、部分正确性和终止性。

## 第五章 依赖类型、结构、子类型与等式

### 5.1 返回类型可以依赖输入

普通函数  $A \rightarrow B$  的返回类型固定为  $B$ ；依赖函数的类型写作

$$(x : A) \rightarrow B(x),$$

返回类型会随输入  $x$  改变。Lean 的全称量词本身就是依赖函数类型。

有限索引是最实用的例子之一：Fin  $n$  表示小于  $n$  的自然数。列表安全索引可以要求一个随列表长度变化的证明：

```
def safeGet (xs : List A) (i : Fin xs.length) : A :=  
  xs.get i
```

调用者不能提供越界的  $i$ ，因此返回类型不需要 Option  $A$ 。代价是程序变换列表长度后，相关索引也可能需要 transport。

### 5.2 无效状态不可表示，还是允许后证明

建模有两种基本路线：

**精确类型** 把长度、边界或合法性编码进类型，使无效状态不可构造；

**普通数据加不变量** 使用简单结构表示状态，再单独定义 Inv  $s$  : Prop 并证明转换保持它。

前者让部分错误在类型检查阶段消失，后者通常更容易编程、重写和对接现有实现。选择标准不是“依赖类型越多越高级”，而是证明成本与接口收益。

### 5.3 structure：命名字段的乘积类型

```
structure StreamState where  
  seen : Nat  
  kept : List Nat  
deriving Repr
```

```
def StreamState.push (s : StreamState) (x : Nat) : StreamState :=
  { seen := s.seen + 1, kept := x :: s.kept }

def StreamState.Inv (s : StreamState) : Prop :=
  s.kept.length <= s.seen
```

结构投影 `s.seen` 和 `s.kept` 是普通函数。更新语法产生新值，不会原地修改旧状态；这使状态转换适合直接写成数学函数。

## 5.4 一个保持不变量的完整小定理

```
theorem StreamState.push_preserves
  (s : StreamState) (x : Nat)
  (h : StreamState.Inv s) :
  StreamState.Inv (s.push x) := by
  simpa [StreamState.Inv, StreamState.push] using
    Nat.succ_le_succ h
```

展开后，目标是

$$\text{length}(x :: s.\text{kept}) \leq s.\text{seen} + 1,$$

即从  $\text{length}(s.\text{kept}) \leq s.\text{seen}$  两边同时加一。这个例子展示了算法形式化的基本结构：状态、转换、不变量、保持性。

## 5.5 Subtype: 值与性质打包

子类型

$$\{x : A \mid P(x)\}$$

包含一个值和它满足  $P$  的证明。例如：

```
def PositiveNat := {n : Nat // 0 < n}

def onePositive : PositiveNat :=
  Subtype.mk 1 (by decide)

#check onePositive.val
#check onePositive.property
```

使用 `subtype` 时，值层计算常可忽略证明字段，但等式和转换有时需要处理 `coercion`。若大量时间都花在搬运证明而不是目标性质上，应重新评估接口。

## 5.6 Sigma 类型与存在命题

依赖对 Sigma B 保存一个索引  $a : A$  和一个类型为  $B(a)$  的数据。它位于 Type，通常用于程序数据；Exists 位于 Prop，主要表达逻辑存在，证明在计算提取中的行为不同。

可以粗略记成：

$\Sigma_{a:A} B(a)$  保存可计算数据， $\exists a : A, P(a)$  声明命题成立。

实际使用时还要检查 universe、proof irrelevance 与是否需要从证明中提取 witness。

## 5.7 等式与 dependent transport

若  $h : a = b$ ，普通重写把关于  $a$  的表达式替换为关于  $b$  的表达式。依赖类型中， $B(a)$  与  $B(b)$  可能是表面不同的类型，需要沿  $h$  搬运数据。这是 Eq.rec、cast、subst 等机制背后的问题。

```
example (a b : Nat) (h : a = b) (P : Nat -> Prop)
  (ha : P a) : P b := by
  cases h
  exact ha
```

对  $h$  做 cases 后只剩反身情形  $a = a$ ，目标退化为已有的  $ha$ 。

### 容易混淆

数学上“长度相等”不代表 Lean 会自动把 `Fin xs.length` 当成 `Fin ys.length`。依赖索引属于类型的一部分。先寻找能避免 transport 的高层 lemma；只有确实需要时再显式搬运。

## 5.8 Typeclass：由类型驱动接口搜索

Typeclass 把操作和定律接口化，实例参数用方括号表示：

```
class HasCost (A : Type) where
  cost : A -> Nat

def totalCost [HasCost A] (xs : List A) : Nat :=
  (xs.map HasCost.cost).sum
```

当调用 `totalCost` 时，elaborator 需要找到 `HasCost A` 实例。代数结构、序、可判定命题、有限类型等 mathlib 接口大量依赖这种搜索。

实例找不到时，应检查：目标类型是否真的是预期类型、实例所在模块是否已导入、是否存在局部歧义、参数有没有被 coercion 改变。不要把所有问题都归因于“mathlib 没这个定理”。



## 5.9 可判定性与计算

要在程序中对命题做 if 分支，Lean 需要 `Decidable p`，即能计算判断  $p$  真假的接口。有限等式、自然数序关系等通常已有实例。`by decide` 可以关闭可计算的可判定命题，但它证明的是当前形式表达式，不会自动选择正确建模。

### 与 AI4Math / AI4TCS 的连接

自动形式化系统经常生成“类型上很强、语义上很怪”的 statement，例如把非法输入排除得过多，使 theorem 变得平凡。依赖类型可以增强规范，也可以隐藏任务偷换；必须把自然语言输入域、Lean 类型和排除条件逐项对照。

### 本章验收

1. 用 `Fin n` 解释安全索引为何不会越界；
2. 比较“精确类型”和“结构加不变量”两种状态建模方式；
3. 编译 `StreamState.push_preserves` 并逐步展开目标；
4. 构造一个 `subtype` 值，分别访问值与证明字段；
5. 说明 `dependent transport` 和普通等式重写有何不同；
6. 诊断一个 `typeclass instance synthesis` 失败的最小例子。

## 第六章 mathlib 检索、自动化与证明调试

### 6.1 先读目标，再搜名字

mathlib 很大，靠猜定理名只能偶尔成功。稳定流程是：

1. 读当前 local context 与 target；
2. 确认对象的具体类型、命名空间和 coercion；
3. 用 #check、文档和源代码找候选定理；
4. 在最小例子中测试候选的参数方向；
5. 最后才把它接回原证明。

```
#check Nat.add_comm
#check Nat.succ_le_succ
#check List.reverse_reverse
#print Nat.add_comm
```

#check 看接口，#print 看声明展开和依赖。阅读库代码时优先寻找同一命名空间中相邻定理，因为命名通常体现对象与操作。

### 6.2 目标形状比自然语言关键词更可靠

自然语言说“单调”，Lean 目标可能是函数单调、严格单调、集合包含或序同态；说“有限”，可能对应 Finset、Fintype、Set.Finite 或有限维空间。先确定形式对象，搜索才不会跨错抽象层。

例如看到

$$a + 1 \leq b + 1$$

应先识别它是自然数序上的 successor 单调，而不是搜索任何含“加法不等式”的定理。

### 6.3 建议型搜索 tactic

mathlib 提供能建议候选脚本的工具：

| 工具                  | 用途                  |
|---------------------|---------------------|
| <code>exact?</code> | 搜索可直接匹配当前目标的项       |
| <code>apply?</code> | 搜索应用后能产生可处理子目标的定理   |
| <code>rw?</code>    | 建议一条可能推进目标的重写       |
| <code>simp?</code>  | 展示建议的简化规则集合         |
| <code>aesop?</code> | 对构造子、假设和安全规则进行结构化搜索 |
| <code>#check</code> | 确认候选定理的完整类型         |
| <code>#print</code> | 查看定义或定理的展开信息        |

问号版本的价值在于返回一段可读建议。把它改写成稳定、较小的显式脚本后，应重新读懂每条依赖；不要长期把宽泛搜索当作唯一证明说明。

## 6.4 常见领域自动化

不同 tactic 解决不同理论：

`norm_num` 具体数字、半环/环中的数值归约；

`ring / ring_nf` 交换半环或环中的多项式恒等式；

`linarith` 由线性等式与不等式组合推出目标；

`nlinarith` 包含有限非线性多项式关系的算术推理；

`omega` Presburger 算术，尤其自然数和整数的线性关系；

`decide` 对有可判定实例且可计算的封闭命题求证；

`native_decide` 借助更高效计算检查可判定命题，使用边界需按项目规范记录。

```
example : (17 : Nat) < 23 := by
  norm_num

example (a b : Int) (h1 : a <= b) : a + 3 <= b + 3 := by
  omega

example (x y : Int) : (x + y)^2 = x^2 + 2*x*y + y^2 := by
  ring
```

自动化失败时先检查问题是否属于它支持的理论。例如 `linarith` 不负责列表归纳，`ring` 不知道任意非环函数的语义。

## 6.5 控制 simp 的作用域

`simp` 依赖一个不断扩展的规则集。学习与稳定证明中常用：

```
simp [myDefinition]
simp only [List.length_cons, Nat.succ_le_succ_iff]
simp [myDefinition] using h
```

`simp only` 让依赖更明确，但可能冗长；普通 `simp` 更易维护，但受全局 `simp` 集变化影响。关键证明可用 `simp?` 查看实际规则，再决定保留哪种粒度。

不要随意把双向都可能使用的定理标记为 `simp`，否则可能造成循环、性能下降或不可预测的规范形。理想 `simp lemma` 把表达式推向明确的正常形。

## 6.6 类型不匹配的诊断

“看起来一样”却无法应用，常见原因包括：

- `Nat` 与 `Int` 之间存在 coercion；
- 一个对象是 `List A`，另一个是 `Finset A`；
- 隐式 `universe` 或类型参数不同；
- 目标经过 `reducible definition` 展开后才匹配；
- 定理需要 `typeclass` 实例，但当前没有或不唯一；
- 等式方向、函数参数顺序或命名空间不对。

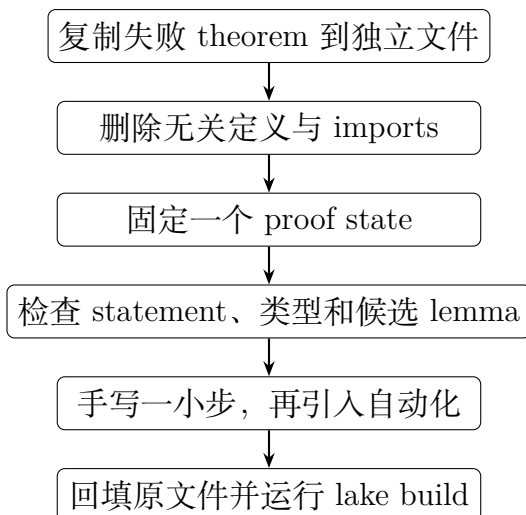
可临时使用更详细打印：

```
set_option pp.all true in
#check Nat.add_comm
```

详细输出很嘈杂，只用于定位隐式结构；问题解决后应恢复可读形式。

## 6.7 最小失败例子

可靠调试流程如下：



记录至少包括：完整错误、当前目标、上下文、尝试过的定理及其 `#check` 输出、最终原因分类。这样的失败轨迹以后可以成为检索数据或 AI proof repair 的训练/评测样本。

## 6.8 不要用过宽 import 掩盖依赖

`import Mathlib` 很适合入门和探索；稳定模块可逐步缩小 imports，以明确依赖和改善构建性能。但过早手工优化 imports 会打断学习，合理顺序是：先让 theorem 在固定版本编译，再用工具或构建错误识别真实依赖。

### 容易混淆

禁止用 `sorry`、额外 `axiom` 或把目标改弱来伪装修复。学习时可以暂时插入 `sorry` 隔离后续错误，但提交前必须在状态记录中显式统计，并区分“文件可编译”和“证明无占位”。

### 与 AI4Math / AI4TCS 的连接

一个有价值的 proof-repair benchmark 不应只记录最终成功脚本，还应保存 Lean/toolchain 版本、原始错误、proof state、允许的依赖和语义不变约束。否则模型可能通过修改 statement 或引入强公理获得虚假成功。

### 本章验收

1. 从目标类型出发，在 mathlib 文档中找到一个所需定理；
2. 分别用 `norm_num`、`omega` 和 `ring` 解决适合它们的问题；
3. 用 `simp?` 查看建议，并改写成一段你能解释的脚本；
4. 把一个失败 theorem 缩成 20 行以内的最小例子；
5. 说明 `sorry`、`axiom`、测试通过和 kernel-checked theorem 的证据差别。

# 第七章 算法不变量、有限随机性与证明分层

## 7.1 先把随机性变成显式输入

普通程序常在函数内部调用随机数生成器，这不利于数学推理。形式化时可以把所有随机选择看成显式输入  $r$ ：

$$\text{run} : \text{Input} \rightarrow \text{RandomTape} \rightarrow \text{Output}.$$

固定  $r$  后， $\text{run } x \text{ } r$  是确定函数；概率来自对 random tape 的分布。这样可以把问题拆成两层：

1. 对每条随机带证明状态安全、输出类型正确等确定性性质；
2. 对随机带集计数或使用概率测度，证明成功事件占多大比例。

### 直觉

随机算法不是“不可预测的代码”，而是“确定程序加一个按指定分布采样的额外输入”。这个视角让执行语义与概率分析分离，也便于用有限枚举、Lean 和外部测试交叉验证。

## 7.2 状态机、不变量与后置条件

流算法可写成：

$$s_0 = \text{init}, \quad s_{t+1} = \text{step}(s_t, x_{t+1}, r_{t+1}), \quad y = \text{query}(s_n).$$

至少区分三类性质：

**表示不变量** 状态字段长度、索引、非负性和内部一致性；

**语义不变量** 状态和已处理输入前缀之间的关系；

**最终保证** 在终止或查询时由不变量推出输出正确性/误差界。

上一章的 `StreamState.Inv` 只保证列表长度不超过已见元素数，它是表示不变量，不说明保留的元素具有正确抽样分布。

### 容易混淆

“状态大小不超过  $k$ ” 不是 reservoir sampling 正确性的全部。真正分布性结论是处理  $n$  个元素后，每个元素以  $k/n$  概率进入样本（在相应条件下），还需要形式化随机选择及其分布。

## 7.3 有限均匀样本空间

入门时可以绕开测度论，令有限类型  $\Omega$  表示所有随机带，均匀概率定义为

$$\Pr[E] = \frac{|\{\omega \in \Omega : E(\omega)\}|}{|\Omega|}, \quad |\Omega| > 0.$$

Lean 中可先定义事件计数：

```
def eventCount [Fintype Omega]
  (E : Omega -> Prop) [DecidablePred E] : Nat :=
  (Finset.univ.filter E).card

def uniformProb [Fintype Omega]
  (E : Omega -> Prop) [DecidablePred E] : Rat :=
  (eventCount E : Rat) / (Fintype.card Omega : Rat)
```

若  $\Omega$  为空，分母为零，必须在定理假设中排除或改用带非空证明的样本空间结构。定义“能通过类型检查”不等于数学接口已完整。

## 7.4 indicator 与期望

事件  $E$  的指示变量

$$\mathbf{1}_E(\omega) = \begin{cases} 1, & E(\omega), \\ 0, & \text{otherwise} \end{cases}$$

满足  $\mathbb{E}[\mathbf{1}_E] = \Pr[E]$ 。有限均匀空间中，这一事实归结为：对所有  $\omega$  求和时，每个满足  $E$  的元素贡献 1，其余贡献 0，因此总和恰为事件基数。

形式化路线可以拆成：

1. 定义可判定事件和 indicator；
2. 证明 indicator 的有限和等于 filter 后的 card；
3. 除以非零的  $|\Omega|$ ；
4. 再把结果包装成期望/概率接口。

这比直接挑战一般概率库中的积分定理更适合作为第一题。

## 7.5 有限 union bound 的计数骨架

对事件  $E_1, \dots, E_m$ , union bound 为

$$\Pr \left[ \bigcup_{i=1}^m E_i \right] \leq \sum_{i=1}^m \Pr[E_i].$$

有限均匀空间下只需证明集合基数不等式

$$\left| \bigcup_i E_i \right| \leq \sum_i |E_i|,$$

再除以  $|\Omega|$ 。重叠元素在右边可能被重复计数, 所以得到上界。Lean 实现时要选择 `Finset`、`Set` 或有限索引族, 并处理 membership、decidability 与 coercion。

## 7.6 随机算法保证的六层拆分

| 层       | 典型命题                 | 合适证据                    |
|---------|----------------------|-------------------------|
| 接口      | 输入可解析、输出类型正确         | 类型检查、编译                 |
| 状态安全    | 索引合法、不变量保持           | Lean 归纳定理               |
| 逐随机带正确性 | 对所有 $r$ 都满足确定性关系     | Lean theorem            |
| 分布正确性   | random tape 确实服从所需分布 | 有限计数或概率库定理              |
| 概率保证    | 失败概率不超过 $\delta$     | 期望、尾界、union bound 的形式证明 |
| 资源保证    | 时间、空间、随机位成本          | 成本语义、复杂度证明或审计分析         |

一个项目可以只形式化其中一部分, 但必须标明其余层是测试、纸笔证明、外部假设还是完全未处理。

## 7.7 Count-Min 的分层形式化示例

Count-Min Sketch 的端到端结论可以先拆成:

1. 数组更新保持所有计数器非负;
2. 目标项每行对应桶包含  $f_i$ , 所以逐行估计不小于  $f_i$ ;



3. 碰撞噪声是其他频率的加权和；
4. 通用哈希给出碰撞概率，推出噪声期望界；
5. Markov 与独立重复给出高概率误差界；
6. 最小值查询和参数  $w, d$  合成最终 theorem。

第一、二步基本不含概率，适合先做；后三步需要哈希族和有限概率接口。完整形式化不是“一条巨大 theorem”，而是一条依赖图。

## 7.8 实现与规范的连接

Lean 中容易证明的纯函数可能与 Python/C++ 高性能实现不同。至少有三种连接策略：

1. 以 Lean 定义为可执行 reference implementation，外部实现做 differential testing；
2. 为外部语言建立语义模型，再证明实现 refinement；
3. 只证明抽象算法 theorem，同时明确工程实现尚未形式化。

第一种最适合个人学习项目。它不能自动证明外部实现正确，但能给测试提供可信 oracle。

### 与 AI4Math / AI4TCS 的连接

AI 生成的随机算法候选应先通过分层 verifier：接口检查、小规模枚举、hidden random tests、确定性不变量证明、概率保证审查和复杂度审查。Lean 最适合其中可精确定义的层，不应被当作覆盖所有研究判断的万能标签。

## 7.9 建议的第一个随机算法形式化题

选择如下梯度：

1. 有限集合上 indicator 的和等于事件基数；
2. 均匀概率总和为 1（显式假设样本空间非空）；
3. 两个或有限多个事件的 union bound；
4. 一个抽样状态的大小不变量；
5. 简化抽样器的无偏性；
6. 最后才进入完整 reservoir sampling、Chernoff 或 Count-Min。

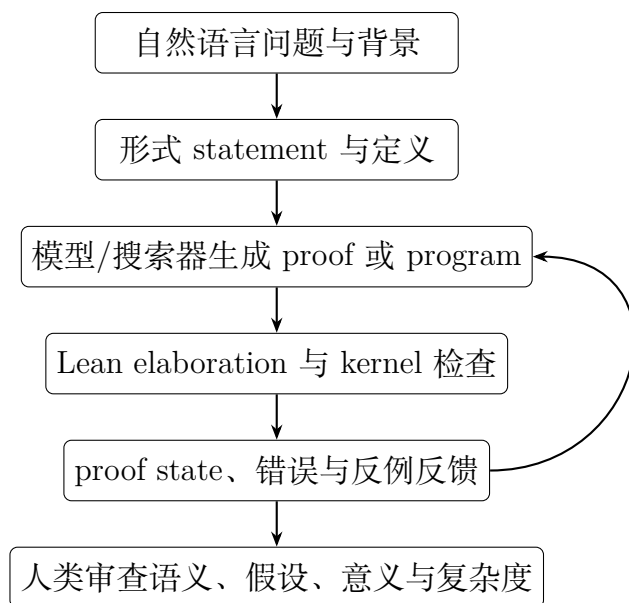
## 本章验收

1. 把随机算法写成输入和 random tape 上的确定函数；
2. 区分表示不变量、语义不变量和分布正确性；
3. 在有限均匀空间中推导 indicator 期望等于事件概率；
4. 写出 union bound 的集合计数证明骨架；
5. 把 Count-Min 端到端保证拆成至少五条 lemma；
6. 为一个只完成部分形式化的项目写清证据边界。

# 第八章 Lean 在 AI4Math 与 AI4TCS 中的证明工程

## 8.1 Lean 在闭环中的位置

一个典型闭环为：



Lean 是强 verifier，不是自动保证研究全流程正确的神谕。它精确回答“这个 proof term 是否证明了这个形式 statement”，其余问题需要额外证据。

## 8.2 AI4Math 的四类任务

**定理证明** statement 已给定，在固定环境中搜索 kernel 可接受证明；

**自动形式化** 把自然语言、公式和上下文翻译成 Lean 定义与 statement；

**proof repair** 给定失败代码和环境，修复证明而保持 statement 语义不变；

**猜想与对象发现** 生成候选定理、反例、构造或数学对象，再经形式/数值工具筛选。

这四类任务的输入、成功条件和风险不同。定理证明 benchmark 上的成功率不能直接代表自动形式化忠实度，更不能代表发现了有价值的新数学。

## 8.3 statement fidelity 是独立任务

自然语言到 Lean 至少要核对：

1. 量词范围：所有对象、存在对象还是某个固定对象；
2. 输入域：自然数、整数、实数、有限图还是任意图；
3. 前提：非空、独立、连通、可测、有限等条件是否遗漏；
4. 等价概念：自然语言术语是否与库定义同义；
5. 结论强度：严格不等式、近似界、概率至少多少；
6. 退化情形：是否通过加入过强假设或空类型让命题平凡成立。

推荐维护三列表：自然语言片段、数学形式、Lean 片段，并对每一处量词和假设做对齐。kernel 无法替你完成这张表。

## 8.4 proof search 的状态、动作与反馈

形式证明搜索可以建模为：

**state** 当前 goals、local context、已导入环境与选项；

**action** tactic、term、lemma application 或 statement decomposition；

**transition** Lean 执行动作后产生新 goals 或错误；

**reward/feedback** 是否编译、目标数量变化、错误类型、证明长度与资源成本；

**terminal** 无 goals 且无占位，kernel 接受最终声明。

仅以“编译通过”为奖励会鼓励系统寻找捷径，例如修改 statement、加入不允许的公理或利用数据泄漏。评测必须冻结目标和允许环境，并审计依赖。

## 8.5 proof repair 的不变约束

一个真正的 proof repair 任务至少固定：

- theorem statement；
- Lean/mathlib 版本；
- 允许修改的代码范围；
- 禁止新增的公理、sorry 和不透明外部假设；
- 资源预算与是否允许网络检索；

- 最终要运行的干净构建命令。

若允许修改 statement，任务就变成联合的 statement repair，需要单独评估语义是否保留。

## 8.6 AI4TCS 中的算法验证

候选算法通常需要多种 verifier：

1. parser/type checker 确认接口；
2. 小规模穷举寻找功能反例；
3. 随机与 OOD 测试检查经验泛化；
4. SMT/SAT 检查有限域或约束性质；
5. Lean 证明不变量、refinement 或数学 theorem；
6. 人类分析时间、空间、随机位和渐近复杂度。

Lean 适合组合已精确定义的性质，但性能结果通常需要成本模型；“函数定义可执行”并不自动给出  $O(n \log n)$ 。

## 8.7 证据登记表

| 证据             | 可以支持                | 不能单独支持               |
|----------------|---------------------|----------------------|
| Lean kernel 接受 | 当前声明在当前环境中可证明       | 翻译忠实、原创、有用、实现高效      |
| 小规模穷举          | 范围内无反例或找到反例         | 任意规模 correctness     |
| 随机测试           | 指定分布与预算下表现          | worst-case guarantee |
| SMT/SAT unsat  | 编码约束下没有反例           | 编码忠实、无限域结论           |
| 基准分数           | 固定 benchmark 上的经验能力 | 开放世界研究能力             |
| 复杂度证明          | 成本模型下的资源上界          | 常数、工程性能、模型外实现        |

## 8.8 数据集与污染问题

形式定理有精确字符串和依赖图，近重复与泄漏尤其容易发生。拆分 benchmark 时应考虑：

- 同一定理的重命名、等价改写和自动生成变体；
- theorem 之间的依赖边，测试定理是否已作为训练 lemma 出现；

- 仓库时间切分与版本历史；
- 自然语言题与形式 statement 的跨语料对应；
- 搜索时访问的文档、检索库和缓存。

随机按 theorem 行拆分通常不足以排除结构泄漏。

## 8.9 失败轨迹也是研究产物

成功 proof 只展示一条路径；失败轨迹包含模型真正缺失的信息。建议保存：

1. 原始 statement 与环境 hash；
2. 每一步 proof state 和候选动作；
3. parser、elaboration、timeout、unknown lemma 等错误分类；
4. 最小失败例子与人工修复理由；
5. 是否修改定义、假设或导入；
6. 最终 proof 的依赖和构建日志摘要。

这些信息可用于错误分析、proof repair、检索改进与可复现实验，而不只是“成功/失败”一位标签。

## 8.10 个人可完成的最小闭环

选择一个 20–50 行的有限对象 theorem：

1. 写自然语言、数学式和 Lean statement 对照；
2. 手写 baseline proof；
3. 人为制造 5 类局部错误；
4. 让模型或搜索工具尝试修复，并保存每次错误反馈；
5. 在隐藏的同构变体上测试；
6. 运行干净 lake build；
7. 报告成功率、失败类型、证明依赖与语义审查结果。

### 容易混淆

不要以“Lean 编译成功”概括整个项目。准确表述应指出：哪个 statement、哪个 toolchain、是否无 sorry/额外公理、自然语言对齐如何检查、实现和复杂度是否在形式化范围内。

### 本章验收

1. 区分 theorem proving、autoformalization、proof repair 与 conjecture discovery；
2. 为一个自然语言题制作 statement fidelity 对照表；
3. 把 proof search 写成 state/action/transition/terminal；
4. 为 proof repair 任务写出不可修改约束；
5. 设计一个 Lean、穷举、随机测试和人工审查分工的 verifier；
6. 解释为什么随机 theorem split 可能产生依赖泄漏。

## 第九章 综合习题与参考答案

### 9.1 习题

- L1. 画出 Lean 源码从 parser/elaborator 到 kernel 的处理路径，并说明 tactic 为什么不必属于可信内核。
- L2. 建立最小 Lake 项目，写出需要提交以复现构建的文件，以及不应提交的缓存目录。
- L3. 定义 `triple : Nat -> Nat`，用 `#eval` 计算一个例子，再证明 `triple n = n+n+n`。
- L4. 分别用 proof term 与 tactic 证明 `p -> q -> And p q`。
- L5. 证明 `Or p q -> Or q p`，解释为什么必须做两个分支。
- L6. 对列表证明 `List.reverse (List.reverse xs) = xs`。若直接搜索 mathlib 引理，记录其完整类型；若手工证明，写出归纳结构。
- L7. 定义一棵二叉树和节点计数函数，证明镜像保持节点数。
- L8. 给出一个归纳假设太弱的例子，写出加强后的 theorem statement。
- L9. 用 `Fin xs.length` 定义安全索引，说明为什么空列表上无法构造索引参数。
- L10. 对 `StreamState.push_preserves` 逐步写出展开前目标、展开后目标和所用自然数引理。
- L11. 构造一个 mathlib theorem 搜索失败的最小例子，按“定义/导入/类型/实例/引理”分类根因。
- L12. 在有限均匀样本空间中证明 indicator 的期望等于事件概率，列出分母非零假设。
- L13. 把 reservoir sampling 的正确性至少拆成状态安全、随机选择分布和最终边缘概率三层。
- L14. 为一个自然语言定理写自然语言、数学式、Lean statement 三列对照，找出至少两个可能的语义丢失点。
- L15. 设计一个 proof-repair 小实验：固定 statement 和 toolchain，定义允许修改范围、五类错误、指标与 hidden 变体。



## 9.2 参考答案与提示

**L1.** 路径为源码/tactic  $\rightarrow$  parser 与 elaborator  $\rightarrow$  带完整隐式信息的核心表达式/证明项  $\rightarrow$  kernel 类型检查。Tactic 只负责构造候选证明项；只要最终项由 kernel 复查, tactic 自身出错通常导致拒绝或产生错误候选, 而不是直接让错误 theorem 被接受。

**L2.** 至少提交 lean-toolchain、Lake 配置、manifest、所有项目源码和构建说明。lake/、编辑器状态和本机缓存不作为源代码提交。复现命令应从干净 checkout 运行 lake build。

**L3.**

```
def triple (n : Nat) : Nat := n + n + n

#eval triple 4

example (n : Nat) : triple n = n + n + n := by
  rfl
```

rfl 成功是因为展开定义后两边相同；#eval 只展示输入 4 的结果。

**L4.** 直接项为 fun hp hq => And.intro hp hq。Tactic 版先 intro hp hq, 再 exact And.intro hp hq, 或者 constructor <;> assumption。应能解释最终项仍是一连串函数与构造子。

**L5.** 对输入证明  $h : Or\ p\ q$  做 cases h。左分支携带 hp : p, 输出 Or.inr hp; 右分支携带 hq : q, 输出 Or.inl hq。析取值可能由任一构造子产生, 漏掉分支就不是总证明。

**L6.** mathlib 已有 List.reverse\_reverse, 应先 #check List.reverse\_reverse 并按当前版本的参数应用。手工证明对 xs 归纳, cons 分支需要 reverse 与 append 的关系; 这也说明复用库 lemma 比从头展开更合适。

**L7.** 按 empty/node 归纳。空树直接 rfl; 节点分支使用左右子树两个归纳假设, 再用自然数加法交换律/结合律处理镜像交换后的顺序。正文 BinTree.size\_mirror 给出参考骨架。

**L8.** 尾递归求和若只证明初始累加器为 0, 归纳分支通常需要对新的累加器  $acc + x$  使用假设。应加强为“对任意 acc, 尾递归结果等于  $acc + \text{sum}(xs)$ ”, 并在归纳时保持 acc 一般。

**L9.** safeGet (xs) (i : Fin xs.length) 的 i 包含一个自然数值与小于 xs.length 的证明。当 xs=[] 时需要构造 Fin 0, 即给出小于 0 的自然数, 这是不可能的。

**L10.** 展开前目标是 `Inv (s.push x)`; 展开后是 `(x::s.kept).length <= s.seen+1`, 化简为 `s.kept.length+1 <= s.seen+1`。由假设和 `Nat.succ_le_succ` 得到, `simp` 只负责连接表达形式。

**L11.** 合格记录要包含最小源码、toolchain、完整错误、proof state、候选定理的 `#check` 和最终分类。若换一个 `import` 就成功, 根因是依赖; 若 `@theorem` 显示参数类型不同, 根因是类型/隐式参数; 若缺 `DecidableEq A`, 根因是实例或缺少假设。

**L12.** 对 `indicator` 在  $\Omega$  上求和, 恰得到满足  $E$  的元素个数。两边除以  $|\Omega|$  得结论。必须假设  $\Omega$  有限且非空, 并明确均匀分布; 事件需要可判定以便有限计算。

**L13.** 状态安全包括样本大小不超过  $k$ 、索引合法; 随机选择分布说明第  $n$  步替换位置的抽样方式; 最终边缘概率通过对  $n$  归纳证明每个已见元素被保留的概率为  $\min(1, k/n)$ 。只完成第一层不能声称 `reservoir sampling` 分布正确。

**L14.** 重点检查量词、输入域、非空/有限/独立等假设、严格或非严格关系, 以及退化输入。例如把“任意连通图”误译成“任意图”, 或额外假设候选解已经最优, 都会改变语义, 即使 Lean 能证明新 `statement`。

**L15.** 固定 `theorem` 文本、`imports` 和 `toolchain`; 只允许改 `proof body`。错误可包括错 `lemma` 名、等式方向、缺失分支、归纳变量错误和实例缺失。报告修复成功率、平均尝试数、错误分类、`proof` 长度、是否引入禁止依赖, 并在变量重命名/同构改写的 `hidden theorem` 上复测。

## 参考文献

- [1] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and contributors. *Theorem Proving in Lean 4*. [https://lean-lang.org/theorem\\_proving\\_in\\_lean4/](https://lean-lang.org/theorem_proving_in_lean4/).
- [2] David Thrane Christiansen. *Functional Programming in Lean*. [https://lean-lang.org/functional\\_programming\\_in\\_lean/](https://lean-lang.org/functional_programming_in_lean/).
- [3] Jeremy Avigad, Patrick Massot, and Floris van Doorn. *Mathematics in Lean*. [https://leanprover-community.github.io/mathematics\\_in\\_lean/](https://leanprover-community.github.io/mathematics_in_lean/).
- [4] The Lean Team. *Lean Language Reference*. <https://lean-lang.org/doc/reference/latest/>.
- [5] The mathlib Community. *mathlib4 Documentation*. [https://leanprover-community.github.io/mathlib4\\_docs/](https://leanprover-community.github.io/mathlib4_docs/).